

نسخه پیشرفته (Advanced Model)

فصل اول: معماری کلی سیستم

۱.۱. رویکرد معماری پیشرفته (Advanced Model)

در مدل پیشرفته این پروژه، بر خلاف مدل عادی که در آن اپلیکیشن به صورت مستقیم با سرویس‌های خارجی نظیر IMDb ارتباط برقرار می‌کند، یک لایه میانی اختصاصی توسعه داده شده است. این امر باعث افزایش امنیت، بهبود سرعت، مدیریت یکپارچه کاربران و امکان سفارشی‌سازی منطق تجاری (Business Logic) می‌شود.

اجزای اصلی معماری عبارتند از:

- ۱. اپلیکیشن موبایل (Client): توسعه یافته با فریم‌ورک Flutter. وظیفه اصلی این لایه، رندر کردن رابط کاربری (UI)، دریافت تعاملات کاربر، اعتبارسنجی اولیه فرم‌ها و ارتباط ایمن با سرور اختصاصی است.
- ۲. سرور اختصاصی (Backend API): این سرور با استفاده از Node.js پیاده‌سازی شده است. وظیفه آن مدیریت احراز هویت، دریافت درخواست‌های کلاینت، برقراری ارتباط با API‌های خارجی (نظیر IMDb / OMDb)، پردازش و کش کردن اطلاعات و پاسخگویی به کلاینت در قالب استاندارد JSON است.
- ۳. پایگاه داده (Database): از طریق ORM معتبر Prisma مدیریت می‌شود و وظیفه ذخیره‌سازی اطلاعات کاربران، لیست‌ها، نظرات، امتیازات و همچنین کش اطلاعات فیلم‌ها را بر عهده دارد.
- ۴. سرویس‌های خارجی (External APIs): سرویس پایگاه داده اینترنتی فیلم‌ها (IMDb) که صرفاً توسط بک‌اند فراخوانی می‌شود.

۱.۲. جریان داده‌ها (Data Flow)

جریان تبادل اطلاعات در سیستم به شرح زیر است:

- درخواست جستجو یا دریافت اطلاعات توسط کاربر در اپلیکیشن ثبت می‌شود.
- درخواست از طریق یک کانال امن (HTTPS) به سرور اختصاصی ارسال می‌گردد.
- سرور ابتدا پایگاه داده محلی خود را بررسی می‌کند (Cache Check).
- در صورت عدم وجود داده، سرور به نمایندگی از کاربر به IMDb درخواست داده، اطلاعات را دریافت، نرمال‌سازی و در دیتابیس خود ذخیره می‌کند.
- در نهایت، سرور پاسخ را با ساختاری استاندارد به اپلیکیشن بازمی‌گرداند.

فصل دوم: طراحی پایگاه داده (Schema Design)

طراحی پایگاه داده بر مبنای پایگاه داده رابطه‌ای (Relational) با استفاده از Prisma Schema شکل گرفته است. در ادامه جزئیات هر یک از مدل‌ها (جداول) آورده شده است:

۲.۱. جدول کاربران (User)

این جدول وظیفه نگهداری اطلاعات هویتی و مدیریت سطوح دسترسی را دارد.

- id: شناسه یکتای کاربر (UUID)
- username: نام کاربری یکتا (Unique)
- email: پست الکترونیکی معتبر و یکتا
- passwordHash: رمز عبور رمزنگاری شده (BCrypt)
- avatarUrl: آدرس تصویر پروفایل کاربر (اختیاری)
- bio: توضیحات کوتاه درباره کاربر (اختیاری)
- role: سطح دسترسی شامل (USER, ADMIN)

۲.۲. جدول رسانه‌ها (Media - کش فیلم و سریال)

جهت کاهش درخواست‌ها به IMDb، اطلاعات رسانه‌ها کش می‌شود.

- id: شناسه یکتا
- imdbId: شناسه یکتا در سیستم IMDb
- title: عنوان اصلی
- type: نوع رسانه (MOVIE یا SERIES)
- posterUrl: آدرس پوستر
- plot: خلاصه داستان
- releaseYear: سال انتشار
- genres: ژانرها (آرایه یا جدول رابط)
- imdbRating: امتیاز در IMDb

۲.۳. جداول مرتبط با سریال (Season & Episode)

- Season: دارای فیلدهای (id, mediaId, seasonNumber)
- Episode: دارای فیلدهای (id, seasonId, episodeNumber, title, releaseDate, runtime, plot)

۲.۴. جدول وضعیت تماشا (WatchStatus)

- userId: شناسه کاربر (رابطه با User)
- mediaId: شناسه رسانه (رابطه با Media)
- status: وضعیت تماشا مقادیر مجاز: (PLAN_TO_WATCH, WATCHING, COMPLETED, PAUSED, DROPPED)

۲.۵. جدول تعاملات (Comment & Rating)

- Comment: فیلدهای (id, userId, mediaId, text, hasSpoiler, createdAt)
- Rating: فیلدهای (userId, mediaId, score (0-100))

فصل سوم: پیاده‌سازی منطق و نیازمندی‌های نرم‌افزار

۳.۱. سیستم پیشرفت تماشای سریال (Watch Progress)

محاسبه پیشرفت تماشا در سرور و کلاینت پیاده‌سازی شده است. منطق رنگ‌بندی به این صورت است: محاسبه درصد: تعداد قسمت‌های علامت‌گذاری شده به عنوان "مشاهده شده" تقسیم بر کل قسمت‌های منتشر شده.

۳.۲. سیستم مدیریت اسپویلر (Spoiler Tagging)

در فرم ثبت نظر، یک چک‌باکس با عنوان "دارای اسپویل" تعبیه شده است. اگر کاربر این گزینه را انتخاب کند، فیلد hasSpoiler در دیتابیس برابر True می‌شود. در اپلیکیشن موبایل، متن چنین نظراتی با یک فیلتر Blur یا پیام "این نظر حاوی اسپویل است، برای مشاهده کلیک کنید" پوشانده می‌شود.

۳.۳. آمار پروفایل کاربر

سیستم قادر است آمار زیر را در پروفایل کاربر تولید کند:

- تعداد فیلم‌های مشاهده شده (با شمارش رکوردهای جدول WatchStatus با نوع Movie و وضعیت Completed).
- مجموع زمان تماشا (با جمع زدن فیلد Runtime قسمت‌ها و فیلم‌های مشاهده شده).
- میانگین امتیازات ثبت شده کاربر.

فصل چهارم: امنیت، احراز هویت و دسترسی‌ها

۴.۱. احراز هویت مبتنی بر توکن (JWT)

برای جلوگیری از حفظ State در سرور و مقیاس‌پذیری بهتر، از توکن‌های JSON Web Token استفاده شده است. پس از لاگین موفق، یک Access Token و یک Refresh Token به کلاینت داده می‌شود. توکن‌ها در حافظه امن گوشی (Secure Storage) ذخیره می‌شوند.

۴.۲. سطوح دسترسی (RBAC)

سیستم دارای سه نوع کاربر کلی است:

- کاربر مهمان (Guest): قابلیت جستجوی فیلم، مشاهده جزئیات، مطالعه نظرات (بدون ارسال)، و مشاهده آثار محبوب. نیازی به توکن ندارد.
- کاربر عضو (Member): قابلیت ثبت‌نام، ورود، ایجاد لیست شخصی، ثبت امتیاز، ثبت نظر و علامت‌گذاری قسمت‌ها.
- مدیر سیستم (Admin): علاوه بر دسترسی‌های عضو، امکان حذف نظرات نامناسب، مشاهده آمار کل سیستم و مسدودسازی کاربران را دارد (بخش ۴.۳ پروژه).

۴.۳. امنیت ارتباطات (Certificate Pinning)

طبق نیازمندی بخش ۸.۳ پروژه، مکانیزم پین کردن گواهینامه (Certificate Pinning) در اپلیکیشن Flutter پیاده‌سازی شده است. با استفاده از پکیج‌های امنیتی، اثر انگشت (Fingerprint) سرور در کلاینت ذخیره شده و اپلیکیشن از اتصال به هر سرور دیگری (حتی با گواهینامه معتبر اما نامرتب) خودداری می‌کند تا حملات شنود (MitM) خنثی شود.

۴.۴. اعتبارسنجی ورودی‌ها (Input Validation)

تمام ورودی‌های سمت کاربر قبل از ثبت در دیتابیس، در لایه سرور اعتبارسنجی می‌شوند. به عنوان مثال:

- رمز عبور باید حداقل ۸ کاراکتر باشد.
- ایمیل باید فرمت استاندارد (Regex) داشته باشد.
- امتیازها باید دقیقاً بین ۰ تا ۱۰۰ باشند.

فصل پنجم: مستندات واسط‌های برنامه‌نویسی (API Documentation)

در این فصل تمامی اندپوینت‌های طراحی شده در بک‌اند با ذکر جزئیات بررسی می‌شوند.

۵.۱. ماژول کاربری و احراز هویت (Auth API)

مسیر: POST /api/auth/register

توضیح: ساخت حساب کاربری جدید.

بدنه درخواست (Body):

```
{
  "username": "ali_reza",
  "email": "ali@example.com",
  "password": "StrongPassword123"
}
```

پاسخ موفق (201 Created):

```
{
  "success": true,
  "data": {"id": "uuid", "username": "ali_reza"}
}
```

مسیر: POST /api/auth/login

توضیح: ورود به حساب و دریافت توکن.

بدنه درخواست (Body):

```
{
  "email": "ali@example.com",
  "password": "StrongPassword123"
}
```

پاسخ موفق (200 OK):

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI...",
  "data": {...}
}
```

۵.۲. ماژول جستجو و دریافت رسانه (Media API)

مسیر: GET /api/media/search?q={query}&type={type}

توضیح: جستجوی فیلم یا سریال. نوع (type) می‌تواند movie یا series باشد.

پاسخ موفق (OK 200):

```
{
  "success": true,
  "data": [
    {
      "imdbId": "tt0944947",
      "title": "Game of Thrones",
      "year": "2011",
      "posterUrl": "https://..."
    }
  ]
}
```

مسیر: GET /api/media/{imdbId}

توضیح: دریافت جزئیات کامل فیلم یا سریال شامل فصل‌ها، بازیگران و نظرات کش شده.

۵.۳. ماژول تعاملات (Interaction API)

مسیر: POST /api/interaction/status

توضیح: ثبت وضعیت تماشا. نیازمند توکن (Bearer Token).

بدنه درخواست:

```
{
  "mediald": "uuid",
  "status": "WATCHING"
}
```

مسیر: POST /api/interaction/comment

توضیح: ثبت نظر روی یک رسانه.

بدنه درخواست:

```
{
  "mediald": "uuid",
  "text": ".فیلم بسیار جذابی بود اما پایان بدی داشت",
  "hasSpoiler": true
}
```

فصل ششم: مدیریت خطاها و پایداری سیستم

۶.۱. استراتژی جامع مدیریت خطا (Error Handling)

همان‌طور که در نیازمندی‌های بخش ۵.۲ و ۷.۹ بیان شده است، سیستم باید در مواجهه با خطاهای مختلف، پیام‌های مناسبی ارائه دهد. بک‌اند از یک فیلتر سراسری (Global Error Handler) برای قالب‌بندی خطاها استفاده می‌کند:

- خطای ۴۰۰ (Bad Request): زمانی رخ می‌دهد که ورودی‌های کاربر نامعتبر باشد (مثلاً ایمیل تکراری).
- خطای ۴۰۱ (Unauthorized): زمانی رخ می‌دهد که توکن منقضی شده یا ارسال نشده باشد.
- خطای ۴۰۳ (Forbidden): زمانی که کاربر عادی قصد دسترسی به API‌های ادمین را داشته باشد.
- خطای ۴۰۴ (Not Found): زمانی که فیلم یا سریال مورد نظر در پایگاه داده یا IMDb یافت نشود.
- خطای ۵۰۳ (Service Unavailable): زمانی که سرویس IMDb پاسخگو نباشد. در این حالت، اگر داده از قبل در کش (پایگاه داده محلی) موجود باشد، سیستم خطای ۵۰۳ نمی‌دهد و اطلاعات کش شده را نمایش می‌دهد.

۶.۲. کشینگ و بهینه‌سازی (Caching Strategy)

بر اساس بخش ۷.۵ پروژه، اطلاعات دریافتی از IMDb در سرور ذخیره می‌شود. چرخه عمر کش (Cache TTL) برای اطلاعات ایستا نظیر "سال انتشار" نامحدود و برای اطلاعات متغیر نظیر "امتیاز IMDb" به صورت دوره‌ای (مثلاً هر ۷ روز یک‌بار هنگام درخواست جدید) به‌روزرسانی می‌شود.

علاوه بر این، تصاویر پوستر (Posters) برای جلوگیری از دانلود مکرر، در سمت کلاینت (فلاتر) با استفاده از ابزارهایی نظیر `cached_network_image` کش می‌شوند تا مصرف حجم اینترنت کاربر (بخش ۸.۸ پروژه) به حداقل برسد.

فصل هفتم: نیازمندی‌های غیرعملکردی

۷.۱. کارایی و عملکرد (Performance)

زمان پاسخ‌دهی سیستم (Response Time) برای صفحات اصلی که از کش خوانده می‌شوند زیر ۵۰۰ میلی‌ثانیه طراحی شده است. استفاده از Pagination (صفحه‌بندی) برای لیست‌های طولانی مانند لیست نظرات و نتایج جستجو باعث می‌شود اپلیکیشن دچار افت سرعت یا مشکل کمبود حافظه (Out of Memory) نشود.

۷.۲. مقیاس‌پذیری (Scalability)

معماری سرور به صورت Stateless طراحی شده است. این یعنی سرور اطلاعات نشست‌ها (Sessions) را در رم خود نگه نمی‌دارد (به لطف استفاده از JWT). بنابراین می‌توان به راحتی سرور را بر روی چندین کانتینر داکر (Docker Containers) اجرا کرد و با یک Load Balancer ترافیک را توزیع نمود.

۷.۳. خوانایی و قابلیت نگهداری کد (Maintainability)

در سمت اپلیکیشن فلاتر از الگوهای مدیریت وضعیت (State Management) استاندارد (همچون Provider یا Bloc) استفاده شده تا منطق UI از منطق دریافت داده کاملاً تفکیک شود. در سمت بک‌اند نیز استفاده از معماری کنترلر/سرویس (Controller-Service-Repository Pattern) رعایت شده است. کدهای پروژه طبق استانداردهای Clean Code نوشته شده و دارای نام‌گذاری‌های استاندارد هستند.

۷.۴. سازگاری (Compatibility)

اپلیکیشن به گونه‌ای طراحی شده (Responsive UI) که روی ابعاد مختلف صفحات نمایش گوشی‌های هوشمند و همچنین نسخه‌های متعدد اندروید (از نسخه ۸ به بالا) بدون بهم‌ریختگی رابط کاربری اجرا شود. پشتیبانی کامل از زبان فارسی و چینش راست‌به‌چپ (RTL) در تمامی صفحات و فرم‌ها پیاده‌سازی شده است.

فصل هشتم: راهنمای استقرار و نتیجه گیری

۸.۱. راه اندازی و استقرار محیط (Deployment)

پروژه شامل فایل های `docker-compose.yml` و `Dockerfile` است که نصب و راه اندازی آن را بسیار سریع و استاندارد می سازد. برای راه اندازی بک اند، مراحل زیر طی می شود:

۱. ایجاد فایل متغیرهای محیطی (`env`) از روی `env.example`.

۲. اجرای دستور `docker-compose up -d` که باعث راه اندازی سرویس های سرور و پایگاه داده در محیط ایزوله می شود.

۳. اجرای دستور `npx prisma migrate dev` جهت ساخت و اعمال جداول پایگاه داده.

۴. اجرای دستور `Seed` برای تزریق داده های اولیه (نظیر ادمین پیش فرض و لیست ژانرها).

۸.۲. خروجی نهایی پروژه

همان طور که در نیازمندی های تحویل پروژه ذکر شده است، پکیج تحویلی شامل موارد زیر خواهد بود:

۱. کدهای منبع سمت سرور (Backend Source Code) به همراه داکر فایل.

۲. کدهای منبع اپلیکیشن (Flutter Source Code).

۳. مستندات فنی و معماری حاضر.

۴. ویدیوهای نمایشی (Demonstration Videos) از امکانات اپلیکیشن و نحوه جستجوی اطلاعات، ثبت نظر، امتیازدهی و رنگ بندی نوار پیشرفت.